

TEKNOLOGI IOT II
Praktikum 2 (LATIHAN 2.6)



DISUSUN OLEH:

NAMA : DEA ANGGRAENI
NPM : 25782036
KELAS :3B
JURUSAN :TEKNOLOGI INFORMASI
PRODI :TEKNOLOGI REKAYASA
INTERNET

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, SPd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026/2027

- **2.6 Latihan**

Latihan 1 (Pemahaman Sistem ADC): Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 1023 (bukan 100 atau 1000)? Kaitkan jawaban Anda dengan penjelasan "Resolusi ADC 10-bit".

Jawaban :

Batas maksimal nilai pembacaan sensor sebesar **1023** disebabkan oleh penggunaan **Resolusi ADC 10-bit** pada sistem perubahan sinyal analog ke digital.

- **Perhitungan Matematis (2^n):**
 - Nilai bit (n) menentukan jumlah kombinasi tingkat diskrit yang dapat dihasilkan oleh ADC menggunakan rumus 2^n .
 - Dengan resolusi **10-bit**, maka jumlah total tingkat diskrit adalah $2^{10} = 1024$ level.
- **Rentang Nilai (0 hingga $2^n - 1$):**
 - Karena sistem digital menghitung angka mulai dari **0**, maka rentang nilai digital yang dihasilkan bergerak dari angka 0 hingga $1024 - 1$, yaitu **0 sampai 1023**.
- **Kesimpulan:**
 - Angka 1023 merupakan representasi nilai digital maksimum dari tegangan referensi penuh yang dapat dibaca oleh ADC beresolusi 10-bit, sehingga angka tersebut bukanlah 100 atau 1000.

Latihan 2 (Implementasi Algoritma): Nilai ADC 0-1023 tidak lazim dibaca oleh pengguna akhir (manusia). Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (rentang 0% hingga 100%). Tuliskan rumus matematika / perintah pembagian yang Anda sematkan di kode tersebut!

Jawaban :

1. **Rumus Matematika / Perintah Pembagian**

Karena ESP32 menggunakan ADC beresolusi 12-bit (rentang nilai **0 hingga 4095**), nilai mentah tersebut harus dikonversi ke dalam bentuk persentase (**0% hingga 100%**). Rumus matematika yang digunakan adalah:

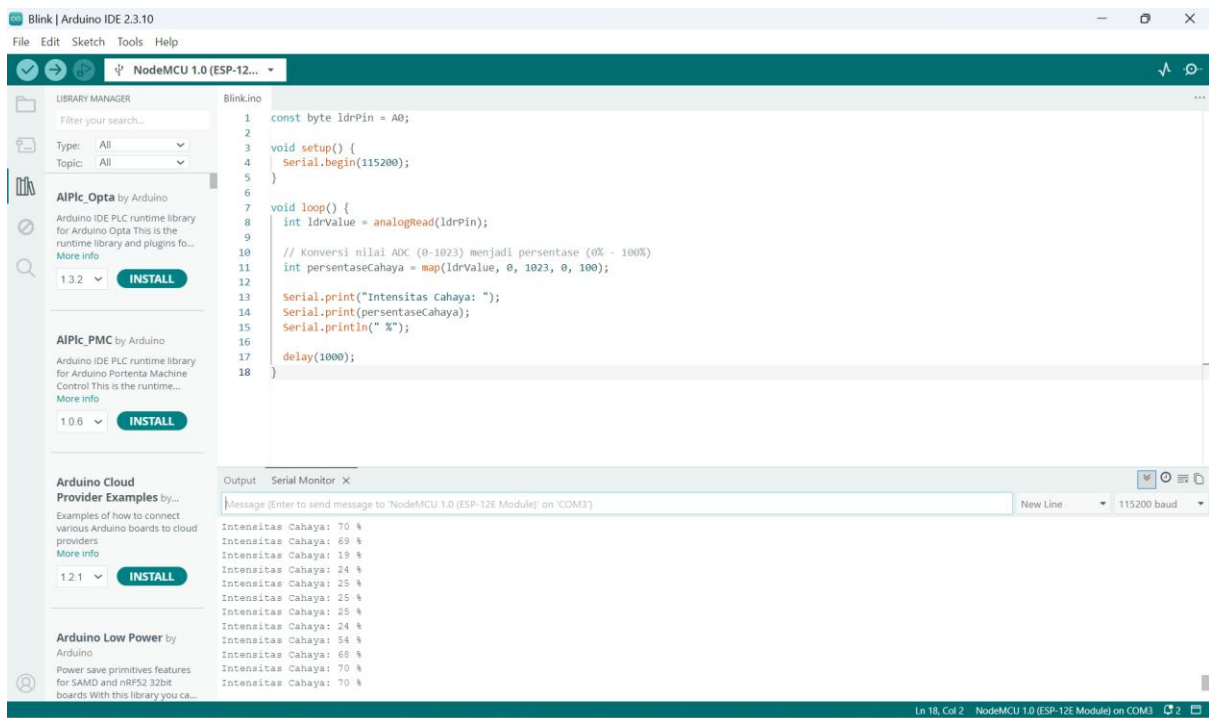
$$\text{Rumus : Persentase} = (\text{Nilai ADC} / 4095) \times 100$$

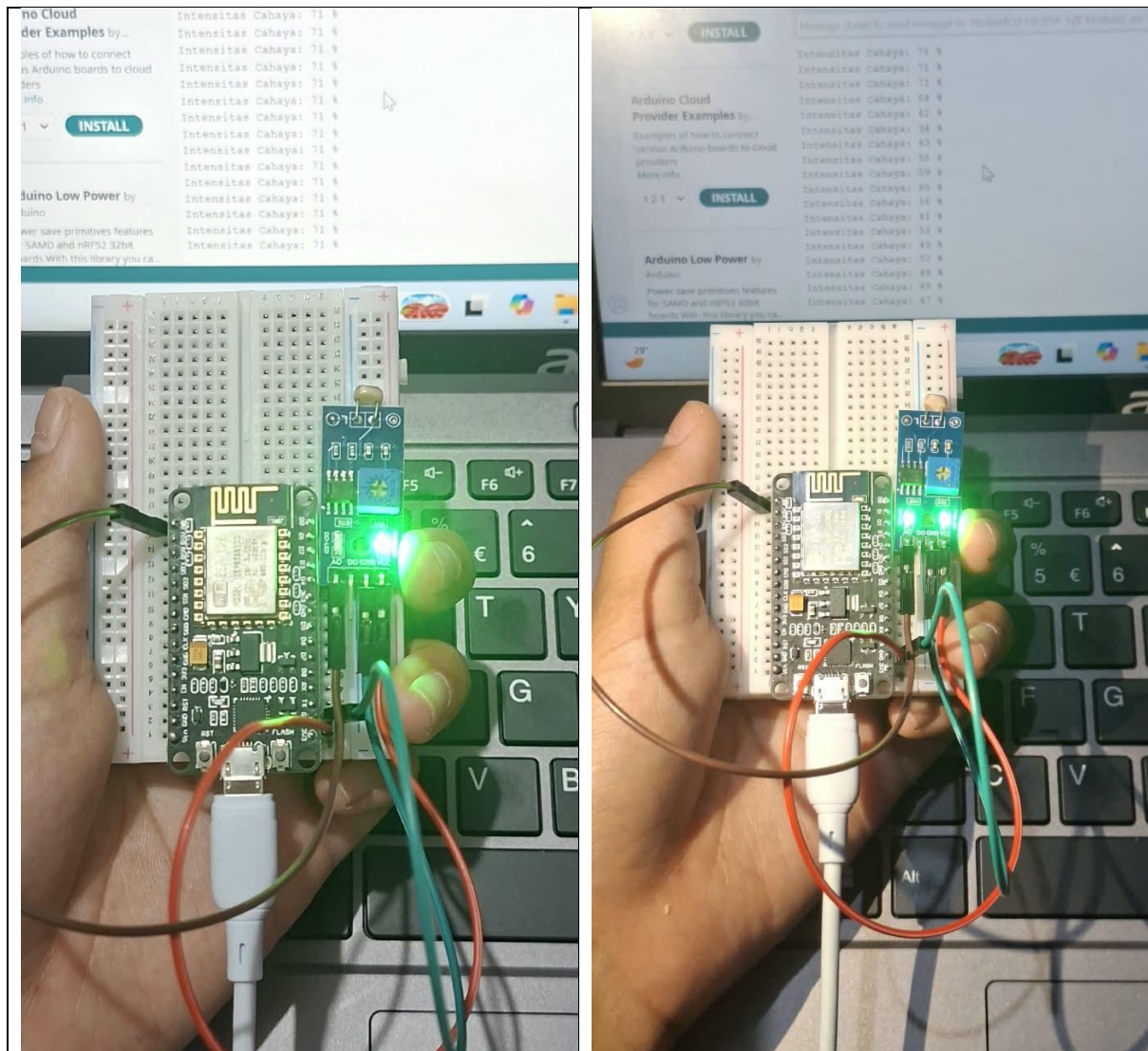
2. **Kode program yang di modifikasi :**

1.0 (ESP-12... ▾

Blink.ino

```
1  const byte ldrPin = A0;
2
3  void setup() {
4      Serial.begin(115200);
5  }
6
7  void loop() {
8      int ldrValue = analogRead(ldrPin);
9
10     // Konversi nilai ADC (0-1023) menjadi persentase (0% - 100%)
11     int persentaseCahaya = map(ldrValue, 0, 1023, 0, 100);
12
13     Serial.print("Intensitas Cahaya: ");
14     Serial.print(persentaseCahaya);
15     Serial.println(" %");
16
17     delay(1000);
18 }
```





Mengubah nilai mentah ADC (0–4095) menjadi skala persentase (0%–100%) agar lebih mudah dipahami manusia. Perhitungan ini juga memperhitungkan karakteristik fisik sensor LDR, di mana **ketika kondisi gelap (tidak ada cahaya), nilai resistansi LDR sangat tinggi sehingga nilai pembacaan ADC-nya tinggi**. Sebaliknya, **ketika terkena cahaya terang, resistansi LDR menurun sehingga nilai pembacaan ADC-nya menjadi rendah**. Oleh karena itu, persentase yang dihasilkan merefleksikan besarnya hambatan/kondisi gelap pada sensor terhadap skala maksimumnya.

Latihan 3 (Analisis Rangkaian): Di subbab 2.3.4 disebutkan sifat modul *Relay* yang terisolasi *Optocoupler*. Banyak *Relay* di pasaran yang bekerja secara **Active-Low**. Jika *Relay* Anda bertipe *Active-Low*, apa yang terjadi pada aktuator (misal: lampu ruangan) yang terhubung ke *Relay* jika program Anda mengeksekusi `digitalWrite(relayPin, HIGH)`?

Jawaban :

Jika program mengeksekusi `digitalWrite(relayPin, HIGH)` pada modul Relay bertipe **Active-Low**, maka aktuator (lampu ruangan) akan **MATI (OFF)**.

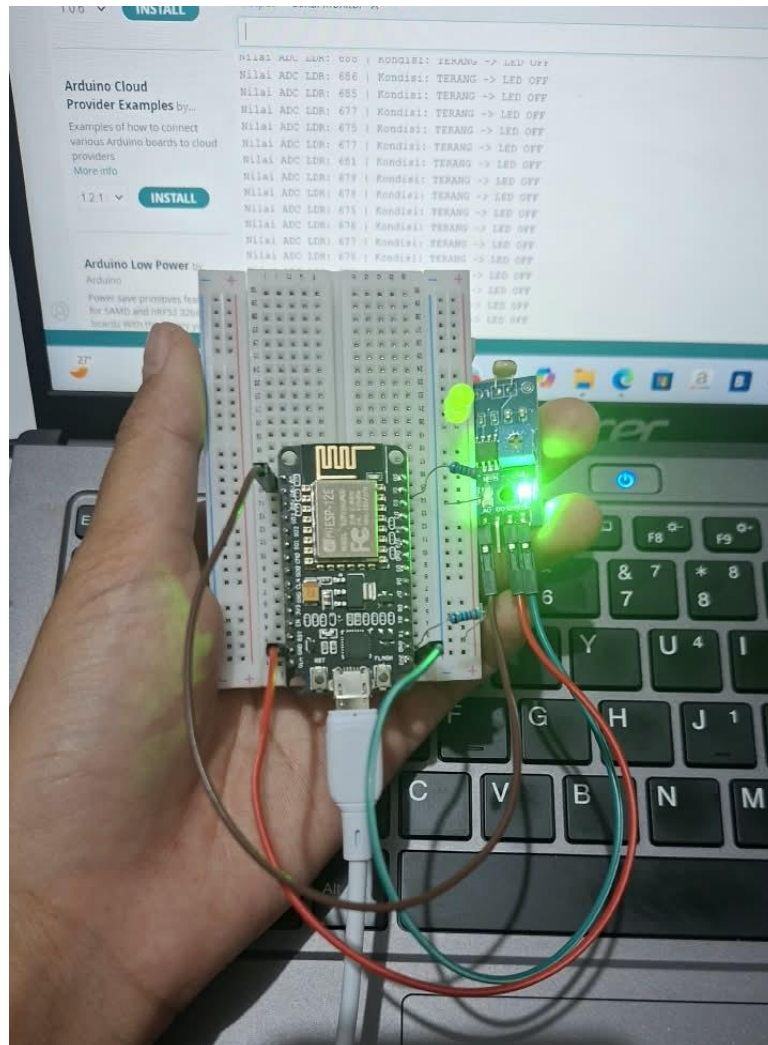
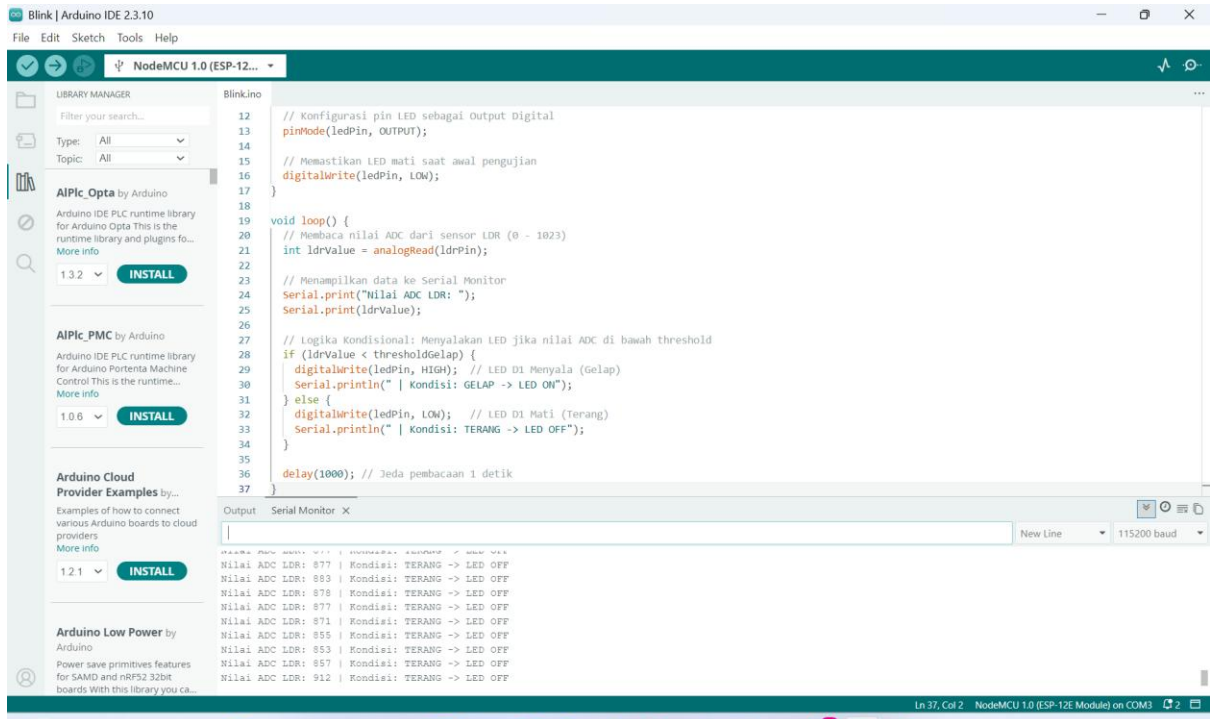
Alasan: Modul Relay *Active-Low* membutuhkan logika LOW (0V) untuk mengaktifkan *optocoupler* dan menutup kontak sakelar. Ketika diberi sinyal HIGH, *optocoupler* tidak terpicu sehingga kontak relay tetap terbuka (*Normally Open*) dan aliran arus ke lampu terputus.

Latihan 4 (Pengembangan Logika): Gabungkan wawasan dari Praktikum 1 Modul 2 ini dengan logika kendali LED dari Praktikum Modul 1. Gunakan blok kondisional `if` yang berfungsi menyalakan sebuah LED (D1) **HANYA JIKA** sensor LDR mendeteksi lingkungan yang gelap gulita (misalnya: ambang batas ADC berada di bawah angka 200).

Jawaban :

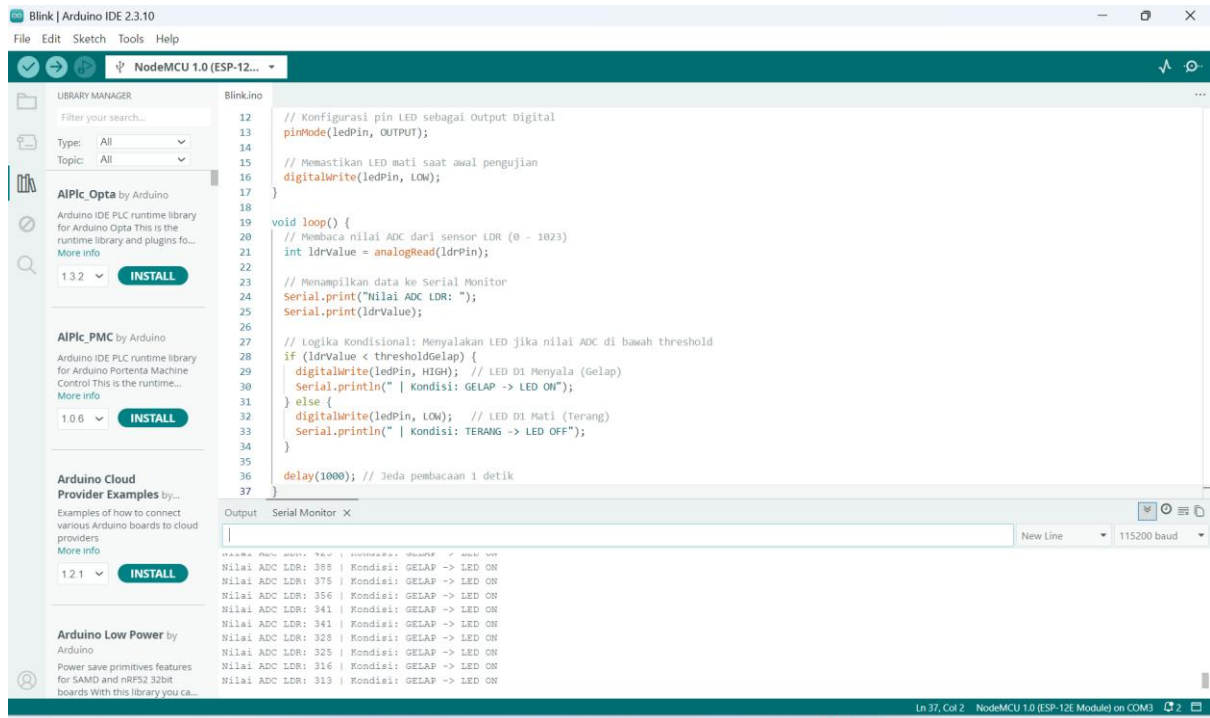
Kode nya :

```
(ESP-12... ▾  
Blink.ino  
2  const byte ldrPin = A0;    // Pin Analog untuk sensor LDR  
3  const byte ledPin = 5;     // Pin D1 pada NodeMCU ESP8266 (GPIO 5)  
4  
5  // Ambang batas disesuaikan ke 600 agar lebih peka  
6  // (tidak perlu menempelkan senter terlalu dekat)  
7  const int thresholdGelap = 600;  
8  
9  void setup() {  
10     Serial.begin(115200);  
11  
12     // Konfigurasi pin LED sebagai Output Digital  
13     pinMode(ledPin, OUTPUT);  
14  
15     // Memastikan LED mati saat awal pengujian  
16     digitalWrite(ledPin, LOW);  
17 }  
18  
19 void loop() {  
20     // Membaca nilai ADC dari sensor LDR (0 - 1023)  
21     int ldrValue = analogRead(ldrPin);  
22  
23     // Menampilkan data ke Serial Monitor  
24     Serial.print("Nilai ADC LDR: ");  
25     Serial.print(ldrValue);  
26  
27     // Logika Kondisional: Menyalakan LED jika nilai ADC di bawah threshold  
28     if (ldrValue < thresholdGelap) {  
29         digitalWrite(ledPin, HIGH); // LED D1 Menyala (Gelap)  
30         Serial.println(" | Kondisi: GELAP -> LED ON");  
31     } else {  
32         digitalWrite(ledPin, LOW);  // LED D1 Mati (Terang)  
33         Serial.println(" | Kondisi: TERANG -> LED OFF");  
34     }  
35  
36     delay(1000); // Jeda pembacaan 1 detik
```



Jika program mengeksekusi `digitalWrite(ledPin, LOW)` saat kondisi lingkungan cukup cahaya (nilai ADC ≥ 1900), maka **LED akan MATI (OFF)**.

Alasannya: Perintah LOW memberikan tegangan 0V (ground) ke pin GPIO 5 ESP32. Karena tidak ada beda potensial atau arus yang mengalir melalui resistor ke katoda LED, komponen LED kehilangan suplai arus sehingga tidak memancarkan cahaya.

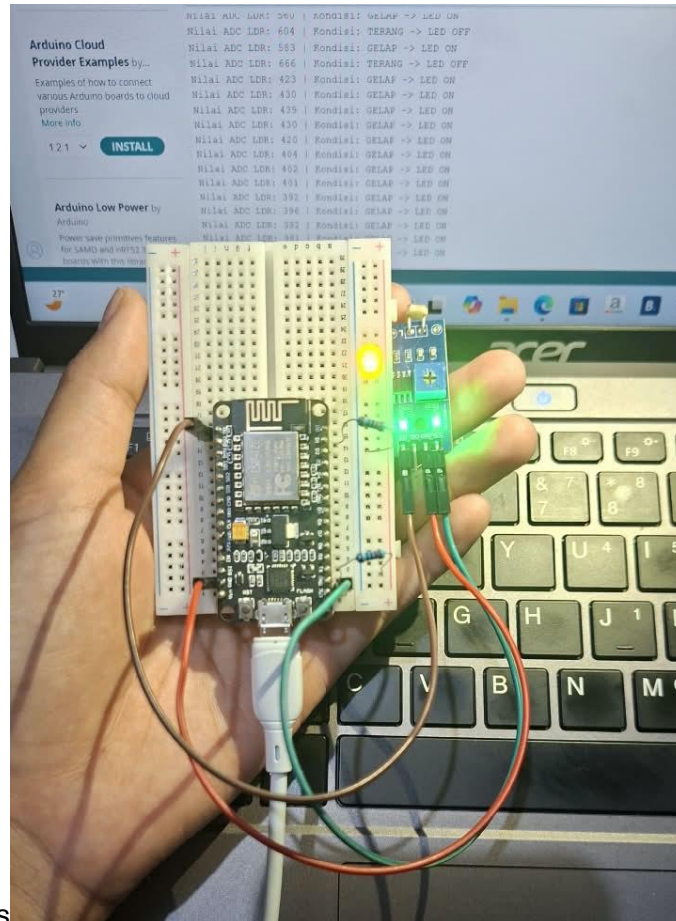


The screenshot shows the Arduino IDE 2.3.10 interface. The main window displays a sketch for an LDR sensor and an LED. The sketch is titled "Blinkino" and is written in C++. The code configures pin 5 as an output, reads the LDR sensor value, and controls the LED based on a threshold. The Serial Monitor shows the output of the code.

```
12 // Konfigurasi pin LED sebagai Output Digital
13 pinMode(ledPin, OUTPUT);
14
15 // Memastikan LED mati saat awal pengujian
16 digitalWrite(ledPin, LOW);
17
18
19 void loop() {
20 // Membaca nilai ADC dari sensor LDR (0 - 1023)
21 int ldrValue = analogRead(ldrPin);
22
23 // Menampilkan data ke Serial Monitor
24 Serial.print("Nilai ADC LDR: ");
25 Serial.print(ldrValue);
26
27 // Logika Kondisional: Menyalakan LED jika nilai ADC di bawah threshold
28 if (ldrValue < thresholdGelap) {
29 digitalWrite(ledPin, HIGH); // LED D1 Menyala (Gelap)
30 Serial.println(" | Kondisi: GELAP -> LED ON");
31 } else {
32 digitalWrite(ledPin, LOW); // LED D1 Mati (Terang)
33 Serial.println(" | Kondisi: TERANG -> LED OFF");
34 }
35
36 delay(1000); // Jeda pembacaan 1 detik
37 }
```

The Serial Monitor shows the following output:

```
Nilai ADC LDR: 388 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 375 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 356 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 341 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 341 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 328 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 325 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 316 | Kondisi: GELAP -> LED ON
Nilai ADC LDR: 313 | Kondisi: GELAP -> LED ON
```



Sebaliknya, jika program mengeksekusi `digitalWrite(ledPin, HIGH)` saat mendapati kondisi gelap (nilai ADC < 1900), maka **LED akan NYALA (ON)**.

Alasannya: Perintah `HIGH` memberikan tegangan keluaran sebesar 3.3V dari pin GPIO 5 ESP32, yang menciptakan jalur aliran arus listrik dari mikrokontroler melalui resistor pengaman menuju ke anoda dan katoda LED, sehingga dioda memancarkan cahaya dengan terang.